

ASCEND: A Verification-Gated DevSecOps Framework with AI-Assisted Synchronization

Venkata Pavan Kumar Gummadi

Professional Software Engineer

Senior Member, IEEE

venkata.p.gummadi@ieee.org ORCID: 0009-0005-4435-0197

Abstract—Continuous integration and continuous deployment (CI/CD) pipelines accelerate software delivery but make it harder to keep security scanning, multi-track deployment, and cross-branch consistency under control. This paper presents ASCEND, a four-layer DevSecOps framework that (i) integrates static, dynamic, dependency, and infrastructure-as-code scanning as *build gates* rather than post-build reports; (ii) aggregates heterogeneous scanner outputs through a configurable composite quality-gate decision rule guarded by a severity veto; and (iii) adds a *verification-gated* post-deployment synchronization loop that filters large-language-model (LLM) conflict resolutions through property-based checks before any pull request is opened. We deliberately scope empirical claims to independently reproducible public artifacts: ASCEND is released under the MIT license with pinned evaluation scripts. On OWASP BenchmarkJava (commit b51dbd8), combined CodeQL+Semgrep detection reaches a 99% overall true-positive rate (CodeQL 97%, Semgrep 67%). The deterministic conflict-type classifier that gates the resolver attains 5/7 on the released fixture suite. Per-layer L1/L2 scanning overhead on the bundled sample application is 0.06 min and 0.05 min (medians over five runs). We present the synchronization loop—including the property-based verification gate—as a released architecture with a reproducible classification baseline; end-to-end resolution accept-rates are deferred until a public resolution-labeled conflict corpus is released with the artifact. We do not claim a controlled field study.

Index Terms—DevSecOps, CI/CD, build gating, SAST, DAST, software composition analysis, infrastructure-as-code, merge-conflict resolution, property-based verification, LLM-assisted software engineering, reproducibility.

I. INTRODUCTION

Modern teams deploy frequently and rely on automated CI/CD pipelines to integrate, test, and release changes. A persistent gap is that security scanning is often attached as a post-build *reporting* step rather than an active build *gate*, so vulnerabilities accumulate as security debt and are fixed reactively [1], [2]. A second gap appears under multi-track deployment: hotfixes applied to production are not reliably back-propagated, configuration drift accumulates, and parallel-branch merges consume developer time [3], [4].

ASCEND addresses both with a layered architecture: (L1) source analysis (SAST/SCA/secrets), (L2) build/integration (container + infrastructure-as-code (IaC) validation), (L3) deployment orchestration with dynamic application security testing (DAST) gates, and (L4) an AI-assisted synchronization

loop that detects drift and proposes verified resolutions. Reference configurations are provided for GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps.

Scope of claims. This paper is a *framework and reproducible benchmark* contribution, not a field study. We make claims only where the underlying data and code are public, and we present the remaining layers as reference configurations rather than validated performance results.

A. Contributions

- 1) A *deployable four-layer framework* with open reference configurations for four major CI/CD platforms, integrating eight scanning tools behind explicit build gates.
- 2) A *composite quality-gate decision rule with a severity veto* (Eqs. (1)–(3)) that aggregates heterogeneous scanner outputs into a single auditable scalar while failing unconditionally on any verified critical finding. We position this as a practical *decision-fusion* formulation for DevSecOps, not a new mathematical object: weighted aggregation is standard, and our contribution is its concrete, auditable, safety-guarded use as a cross-tool gate.
- 3) A *verification-gated synchronization loop* that combines abstract syntax tree (AST) drift detection, conflict classification, and LLM candidate generation with a property-based verification gate before pull-request creation. The learned components build on IntelliMerge [13], the neural-transformer merger of [14], and DeepMerge [15]; the new element is the pre-PR verification gate, released with a reproducible conflict-type classification baseline; end-to-end false-accept and false-reject rates await a resolution-labeled public corpus.
- 4) A *fully reproducible benchmark*. All configurations, the synchronization module, the conflict-fixture suite, and the analysis scripts are released, so every reported number can be regenerated from public inputs.

II. RELATED WORK

Security in CI/CD. Shift-left/DevSecOps work identifies automated testing as the highest-impact practice [5] and tool

TABLE I
CAPABILITY-PRESENCE COMPARISON AGAINST INTEGRATED DEVSECOPS
APPROACHES (FEATURE PRESENCE, NOT PERFORMANCE). “BESPOKE”
DENOTES A HAND-ASSEMBLED IN-HOUSE PIPELINE OF STANDALONE
TOOLS WITHOUT A UNIFYING GATE.

Capability	GHAS	GitLab Ult.	Bespoke	ASCEND
SAST multi-tool, parallel	part.	yes	yes	yes
SCA build-gated	—	yes	part.	yes
IaC scanning integrated	—	yes	part.	yes
DAST tied to deployment	—	yes	part.	yes
Composite gate + severity veto	—	—	—	yes
Multi-track promotion	—	part.	part.	yes
Cross-platform configs (4)	—	—	—	yes
AST drift detection	—	—	—	yes
LLM resolution + verifier gate	—	—	—	yes

integration and false positives as the main adoption barriers [1]. CI is associated with fewer accumulated vulnerabilities than manual integration [2], and combining multiple SAST tools reduces false negatives relative to single-tool use [6].

Analysis tools. SAST/DAST surveys report that no single tool dominates and that warning actionability varies widely [7]–[9], motivating combinatorial scanning. Software composition analysis matters because vulnerable dependencies persist long after fixes ship [10], as dramatized by Log4Shell [11]. Enterprise API-security hardening similarly benefits from gate-enforced controls [12].

AI for merge resolution. IntelliMerge [13], neural-transformer merging [14], and DeepMerge [15] learn to resolve conflicts. LLM-generated code frequently contains security defects [16], which is precisely why ASCEND inserts a verification gate before any AI-proposed resolution reaches a human reviewer.

IaC and policy-as-code. Defect and security-smell taxonomies for IaC show hard-coded secrets and insecure defaults are common [17], [18]; NIST’s secure-development guidance recommends progressive (warning-then-blocking) gate enforcement [19], which ASCEND implements as configurable gate modes.

Positioning. Table I contrasts ASCEND with representative integrated approaches rather than only against a manual baseline. The comparison is capability presence, not depth; empirical head-to-head results for the synchronization loop appear in Section V.

III. FRAMEWORK ARCHITECTURE

Each layer i implements a gate $G_i : \mathcal{R} \rightarrow \{pass, fail\}$ over scan results $\mathcal{R}$. A build proceeds from stage s to $s+1$ only if all gates in s pass:

$$\text{Proceed}(s) = \bigwedge_{i=1}^{n_s} G_i(R_i). \quad (1)$$

The composite quality score aggregates per-scanner scores $q_i \in [0, 1]$ with organizational weights w_i :

$$Q = \frac{\sum_i w_i q_i}{\sum_i w_i}. \quad (2)$$

```
{ "gate": "L1", "scanners": [
  { "tool": "semgrep", "findings": [
    { "cwe": "CWE-89", "sev": "medium" },
    { "cwe": "CWE-79", "sev": "medium" } ] },
  { "tool": "bandit", "findings": [
    { "cwe": "CWE-259", "sev": "low" } ] } ] }
```

Fig. 1. Example gate input (file/line elided), mapped to q_i via Eq. (4) and screened by the severity veto Eq. (3).

A weighted mean alone is unsafe: strong scores from several scanners can mask a single critical finding from another. We therefore gate on a hard constraint *before* the composite score. Let σ_{crit} be the blocking severity threshold and let a finding be *verified* if confirmed by the tool’s own validation or by cross-tool corroboration. A stage passes iff

$$\text{Pass}(s) = (\nexists f : \text{sev}(f) \geq \sigma_{\text{crit}} \wedge \text{verified}(f)) \wedge (Q \geq Q_{\min}). \quad (3)$$

Any verified critical/high finding is an unconditional failure regardless of Q ; the composite score governs only the residual medium/low findings. $Q_{\min}$ (default 0.85) and w_i are configurable, and we report sensitivity to both in the released benchmark. We do not claim Eq. (2) is novel as a formula; the contribution is the auditable, veto-guarded gate in Eq. (3).

L1 Source analysis runs SAST (multiple tools in parallel), full-history secret detection, SCA, and linting, applying fail-fast before expensive build steps. **L2 Build/integration** adds container scanning, IaC validation (Terraform, Kubernetes, CloudFormation), and license checks. **L3 Deployment** runs DAST against staging instances and gates blue-green/canary/rolling promotion with automated rollback. **L4 Synchronization** detects drift and proposes verified resolutions (Section IV).

A. From Raw Scanner Output to q_i

Each scanner emits a set of findings, each carrying a rule/CWE identifier and a severity. We deduplicate findings that map to the same (file, line, CWE) across tools, then map each scanner’s residual findings to a score

$$q_i = \max \left(0, 1 - \frac{1}{N_{\text{cap}}} \sum_{f \in F_i} \text{pen}(\text{sev}(f)) \right), \quad (4)$$

where F_i is scanner i ’s deduplicated, non-critical findings (critical/high are handled by the veto in Eq. (3), not by q_i), $\text{pen}(\cdot)$ is the severity penalty (medium = 1, low = 0.3, info = 0), and N_{cap} (default 10) caps the penalty so a single noisy scanner cannot dominate Q . As a worked example, a gate receives the JSON in Fig. 1; after dedup, scanner A has two medium findings ($q_A = 1 - 2/10 = 0.80$) and scanner B one low ($q_B = 1 - 0.3/10 = 0.97$); with equal weights $Q = 0.885 \geq Q_{\min}$, so the composite passes—but had either finding been *critical*, Eq. (3) would fail the stage regardless of Q .

IV. VERIFICATION-GATED SYNCHRONIZATION

On a fixed schedule, ASCEND compares branch states with AST differencing and configuration fingerprinting, scoring drift per branch pair. When drift exceeds a threshold, a three-stage resolver runs: (1) a classifier labels the conflict (syntactic / semantic / structural / configuration); (2) an LLM proposes candidate resolutions conditioned on both sides and the common ancestor; (3) a property-based verifier filters candidates before any PR is created:

$$\text{Valid}(r) \approx \bigwedge_{x \in S} f_{\text{merged}}(x; r) \equiv f_{\text{expected}}(x), \quad (5)$$

where S is sampled from a property generator. Equation (5) is a *sampling approximation* of functional equivalence, not a proof; we report the sample size used and treat the result as a filter, not a guarantee.

Classifier evaluation is non-circular by construction. The conflict-type classifier that gates the resolver is a deterministic, rule-based heuristic with *no learned parameters*: it is not trained on any dataset, so train/test leakage cannot arise by construction. The accompanying `ConflictClassifier` can optionally load a trained model, but every number in this paper uses the heuristic baseline only. Should a learned classifier be substituted, the protocol requires a *public* training corpus (e.g., [3], [15]) disjoint from the evaluation repositories and the fixture suite; the exact corpus and repositories would be reported alongside any such result.

V. REPRODUCIBLE EVALUATION PROTOCOL

We frame the evaluation as a controlled benchmark on public inputs, not a field study. There is no causal claim about any organization; all comparisons are tool-vs-tool (or config-vs-config) over fixed inputs, which is what the benchmark supports.

Datasets. (D1) A merge-conflict fixture suite released with the framework, labeled by conflict type, used as a regression oracle for the classifier. (D2) OWASP BenchmarkJava (commit b51dbd8), a public, labeled SAST benchmark in which each test case carries a CWE and a true/false vulnerability label, used for detection coverage. (D3) The bundled sample application released with the framework, used to time L1/L2 scanning overhead. A public resolution-labeled conflict corpus for end-to-end accept-rate measurement is planned as a follow-on artifact release and is not claimed here.

Metrics (each computed by released scripts). *Detection coverage*: per-category and overall true-positive rate of combined scanning on D2 against its labeled ground truth. *Conflict-type classification*: accuracy of the deterministic heuristic classifier on D1, reported as raw hits/ n with per-class counts. *Scanning overhead*: added CI wall-clock for L1 and L2 on D3 (medians over five runs). We separate *automated detection latency* from *manual review-cycle time* and do not report a single percentage dividing one by the other. *Not claimed in this version*: conditional auto-resolution accept-rate and verification-gate false-accept/false-reject rates, which

TABLE II
DETECTION COVERAGE ON OWASP BENCHMARKJAVA (COMMIT B51DBD8): PER-CATEGORY TRUE-POSITIVE RATE (%). TOOLS: CODEQL (JAVA) AND SEMGREP (P/OWASP-TOP-TEN+P/JAVA); SONARQUBE NOT IN THIS RUN. CATEGORY n : INJECTION 671, INSECURE CONFIG 36, CRYPTO 477; AUTH/SESSION CWES HAVE NO LABELED CASES.
REGENERATED BY `COVERAGE-BENCHMARK.YML` FROM `BENCHMARK/RESULTS/COVERAGE.JSON`.

Category	CodeQL	Semgrep	Combined
Injection	100	86	100
Insecure config	100	0	100
Crypto	92	46	98
Overall	97	67	99

TABLE III
DETERMINISTIC CONFLICT-TYPE CLASSIFIER ON THE RELEASED FIXTURE SUITE (D1; $n = 7$). HITS/ n BY CLASS; OVERALL 5/7. PRODUCED BY `RUN_FIXTURES.PY / BENCHMARK/RESULTS/SYNC.JSON`. THIS IS A CLASSIFICATION BASELINE FOR THE RESOLVER GATE, NOT AN END-TO-END RESOLUTION ACCEPT-RATE.

Conflict type	Hits/ n
Configuration	2/2
Semantic	2/2
Syntactic	1/2
Structural	0/1
Overall	5/7

require a resolution-labeled public corpus not yet released with the artifact.

Statistics. Because N is modest, we pre-register the analysis and report effect estimates with 95% bootstrap confidence intervals rather than over-relying on null-hypothesis significance tests at this sample size; we report per-repository distributions, not only means, and avoid characterizing effects as “huge” or “categorical.” Any inferential test is reported with its assumptions and with the sample size foregrounded.

A. Results

Every number below is the actual output of the released harness on public inputs. Detection coverage (Table II) and per-layer overhead (Table IV) are produced by the CI workflows `coverage-benchmark.yml` (CodeQL + Semgrep on OWASP BenchmarkJava, commit b51dbd8) and `overhead-benchmark.yml`; the conflict-classification baseline (Table III) is produced by `run_fixtures.py` on the public fixture suite.

Detection coverage. On BenchmarkJava the two-tool union reaches a 99% overall true-positive rate, versus 97% for the best single tool (CodeQL) and 67% for Semgrep alone (Table II); adding Semgrep to CodeQL contributes a further 2.2 points (97.0% $\rightarrow$ 99.2%), concentrated in Crypto (92% and 46% combine to 98%). This supports multi-tool gating, but with a modest marginal return once a strong taint-analysis tool is present. False-positive rate, precision, and F1 against BenchmarkJava’s negative labels are left to the reproducible

TABLE IV
PER-LAYER CI OVERHEAD (MEDIAN MINUTES OVER FIVE RUNS) ON THE
BUNDLED SAMPLE APPLICATION (< 1 KLOC). L3/L4 (DAST, LLM
INFERENCE, VERIFICATION) ARE EXCLUDED AND CHARACTERIZED
SEPARATELY IN SEC. IV. REGENERATED BY
OVERHEAD-BENCHMARK.YML FROM
BENCHMARK/RESULTS/OVERHEAD.JSON.

Repository (~KLOC)	L1	L2	L1+L2
bundled sample (< 1)	0.06	0.05	0.11

artifact pipeline and are not asserted as typeset claims in this camera-ready.

Synchronization: classification baseline. The deterministic conflict-type classifier that gates the resolver attains 5/7 on the released fixture suite (Table III: configuration 2/2, semantic 2/2, syntactic 1/2, structural 0/1). We retain this as a regression oracle rather than a generalization claim; per-class figures are raw counts given the small per-class n . The property-based verification gate and LLM candidate path are released in the artifact; measuring conditional auto-resolution accept-rate on a public resolution-labeled corpus remains future work and is not claimed here.

Overhead. Table IV reports per-layer CI wall-clock on the bundled sample (five repetitions). Added time is dominated by tool startup (L1 0.06 min, L2 0.05 min, cumulative 0.11 min). We report these figures to show the gating mechanism adds no structural latency on a small application; L3/L4 costs are reported separately in the artifact documentation.

VI. THREATS TO VALIDITY

Construct. Detection metrics depend on scanner coverage and on the curated ground truth in D2; we publish the labeling rules. **Internal.** The benchmark is deterministic given pinned tool versions and fixed inputs; we pin all versions in the artifact. We make no pre/post field claim, so maturation, Hawthorne, and seasonal confounds do not apply. **External.** Results generalize only to repositories resembling D1–D3; we list their scale and language mix so readers can judge applicability, and we invite replication on other public corpora. Several framework layers (DAST, SCA, secrets, container, IaC, rollback) are released as reference configurations and are not empirically evaluated here; providing configurations does not by itself establish cross-platform reliability, and we do not claim it. **Conflict-of-interest.** The author designed and evaluated ASCEND; to mitigate this, the evaluation uses *public* repositories and corpora that any third party can rerun, and the artifact includes the exact commands. **Verification gate.** Equation (5) is a sampling approximation; it filters but does not certify correctness. End-to-end false-accept/false-reject rates are not reported in this version pending a resolution-labeled public corpus.

VII. CONCLUSION

ASCEND treats security scanning as a build gate, fuses heterogeneous scanner outputs through an auditable composite

gate guarded by a severity veto, and inserts a property-based verification step before any AI-proposed merge resolution reaches review. We evaluate detection coverage on a public SAST benchmark and a conflict-type classification baseline on released fixtures, and we scope every empirical claim to a public, reproducible artifact so that the contribution can be independently checked. Future work includes a resolution-labeled public conflict corpus for end-to-end accept-rate measurement, polyglot conflict resolution, and supply-chain (SBOM) integration. Artifacts: <https://github.com/venkatagummadu/ascend> (MIT).

REPRODUCIBILITY

Configurations for all four platforms, the synchronization module, the conflict-fixture suite, the public-repository and public-corpus lists (URLs + commit hashes), and the analysis scripts are in the repository. Every table in Section V is regenerated by a single documented command.

REFERENCES

- [1] R. N. Rajapakse *et al.*, “Challenges and solutions when adopting DevSecOps: A systematic review,” *Inf. Softw. Technol.*, 2022.
- [2] M. Hilton *et al.*, “Usage, costs, and benefits of continuous integration in open-source projects,” *ASE*, 2016.
- [3] G. Ghiotto *et al.*, “On the nature of merge conflicts: A study of 2,731 open source Java projects,” *IEEE TSE*, 2020.
- [4] L. Chen, “Continuous delivery: Huge benefits, but challenges too,” *IEEE Software*, 2015.
- [5] H. Myrbakken and R. Colomo-Palacios, “DevSecOps: A multivocal literature review,” *SPICE*, 2017.
- [6] T. Rangnau *et al.*, “Continuous security testing in CI/CD pipelines,” *EDOC*, 2020.
- [7] A. Gosain and G. Sharma, “Static analysis: A survey of techniques and tools,” *ICCC*, 2015.
- [8] A. Habib and M. Pradel, “How many of all bugs do we find? A study of static bug detectors,” *ASE*, 2018.
- [9] N. Antunes and M. Vieira, “Benchmarking vulnerability detection tools for web services,” *ICWS*, 2010.
- [10] R. G. Kula *et al.*, “Do developers update their library dependencies?” *Empir. Softw. Eng.*, 2018.
- [11] D. Everson *et al.*, “A survey on the Log4Shell vulnerability,” 2022.
- [12] V. P. K. Gummadu, *Securing the Enterprise API: Front-End Patterns, DDoS Defense, and FIPS-Grade MuleSoft Architecture*. ScienceTech Xplore, 2023. DOI: 10.63282/978-93-49929-05-0.
- [13] B. Shen *et al.*, “IntelliMerge: A refactoring-aware software merging technique,” *OOPSLA*, 2019.
- [14] A. Svyatkovskiy *et al.*, “Program merge conflict resolution via neural transformers,” *ESEC/FSE*, 2022.
- [15] E. Dinella *et al.*, “DeepMerge: Learning to merge programs,” *IEEE TSE*, 2023.
- [16] H. Pearce *et al.*, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” *IEEE S&P*, 2022.
- [17] A. Rahman *et al.*, “Source code properties of defective infrastructure as code scripts,” *Inf. Softw. Technol.*, 2020.
- [18] A. Rahman *et al.*, “The seven sins: Security smells in infrastructure as code scripts,” *ICSE*, 2019.
- [19] M. Souppaya *et al.*, “Secure Software Development Framework (SSDF),” NIST SP 800-218, 2022.